Solution: Can Place Flowers

Problem Statement

You have a long, narrow flowerbed divided into sections, each of which can either be empty or filled with a flower. Due to gardening restrictions, you cannot plant flowers in `adjacent sections`—they must be at least one section apart to prevent overcrowding.

Given the `current state` of the flowerbed (represented as an array, where `0` indicates an empty section and `1` signifies a section with a flower) and a number representing how many more flowers you wish to plant, determine if you can plant all the desired flowers without `breaking` the gardening restrictions.

Examples

- **Example 1**:
  - **Input**: flowerbed = `[0,0,1,0,1,0,0]`, n = `2`
  - **Expected Output**: true
  - **Justification**: You can plant flowers in the 0th and 7th index, satisfying the non-adjacent rule.
- **Example 2**:
  - **Input**: flowerbed = `[1,0,0,0,0,1]`, n = `2`
  - **Expected Output**: false
  - **Justification**: Despite the four consecutive empty sections, you can only plant one flower (either in the 2nd or 3rd index) without violating the rule of not planting in adjacent sections.
- **Example 3**:
  - **Input**: flowerbed = `[0,0,1,0,0]`, n = `1`
  - **Expected Output**: true
  - **Justification**: You can plant one flower either in the 0th or 4th index, adhering to the non-adjacent rule.

Solution

To solve this problem, the strategy involves iterating through the flowerbed array while keeping track of empty sections that are eligible for planting. We need to consider three main scenarios: the beginning of the array, the middle sections, and the end of the array. The idea is to find consecutive empty sections (`0`s) and determine if a flower can be planted according to the rules. A flower can be planted in a section if it and its immediate neighbors (if they exist) are empty. This approach works because it directly assesses each potential planting spot by its immediate surroundings, ensuring compliance with the non-adjacency requirement.

The reason this strategy is effective is that it allows for a sequential assessment of the flowerbed, minimizing the need for backtracking or complex conditionals. By treating the flowerbed as a series of potential spots for planting, we can incrementally adjust our count of how many more flowers need to be planted. This method is efficient because it only requires a single pass through the array, ensuring that each section is considered exactly once, and makes decisions based on the current state without needing to revisit or revise earlier decisions.

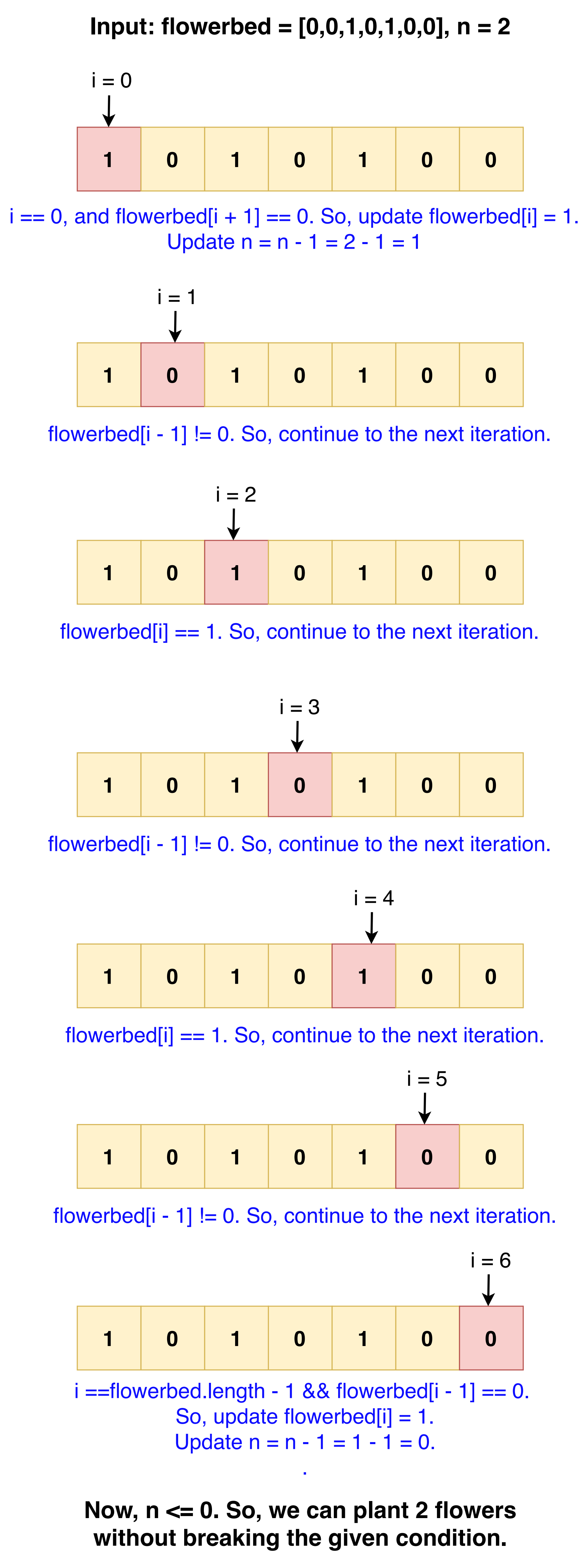
Step-by-step Algorithm

- Initialize a count of how many flowers you need to plant (`n`).
- Iterate through each section of the flowerbed.
  - For each section, check if it is empty.
    - If the section is at the beginning or end of the flowerbed, only consider one neighbor.
    - If the section is in the middle, consider both neighbors.
  - If the current section and its applicable neighbors (if any) are empty, plant a flower there (mark the section as `1`) and decrease the count of flowers needed to plant by one.
- After the loop, if the count of flowers needed to plant is 0 or less, return `true` indicating you can plant all desired flowers. Otherwise, return `false`.

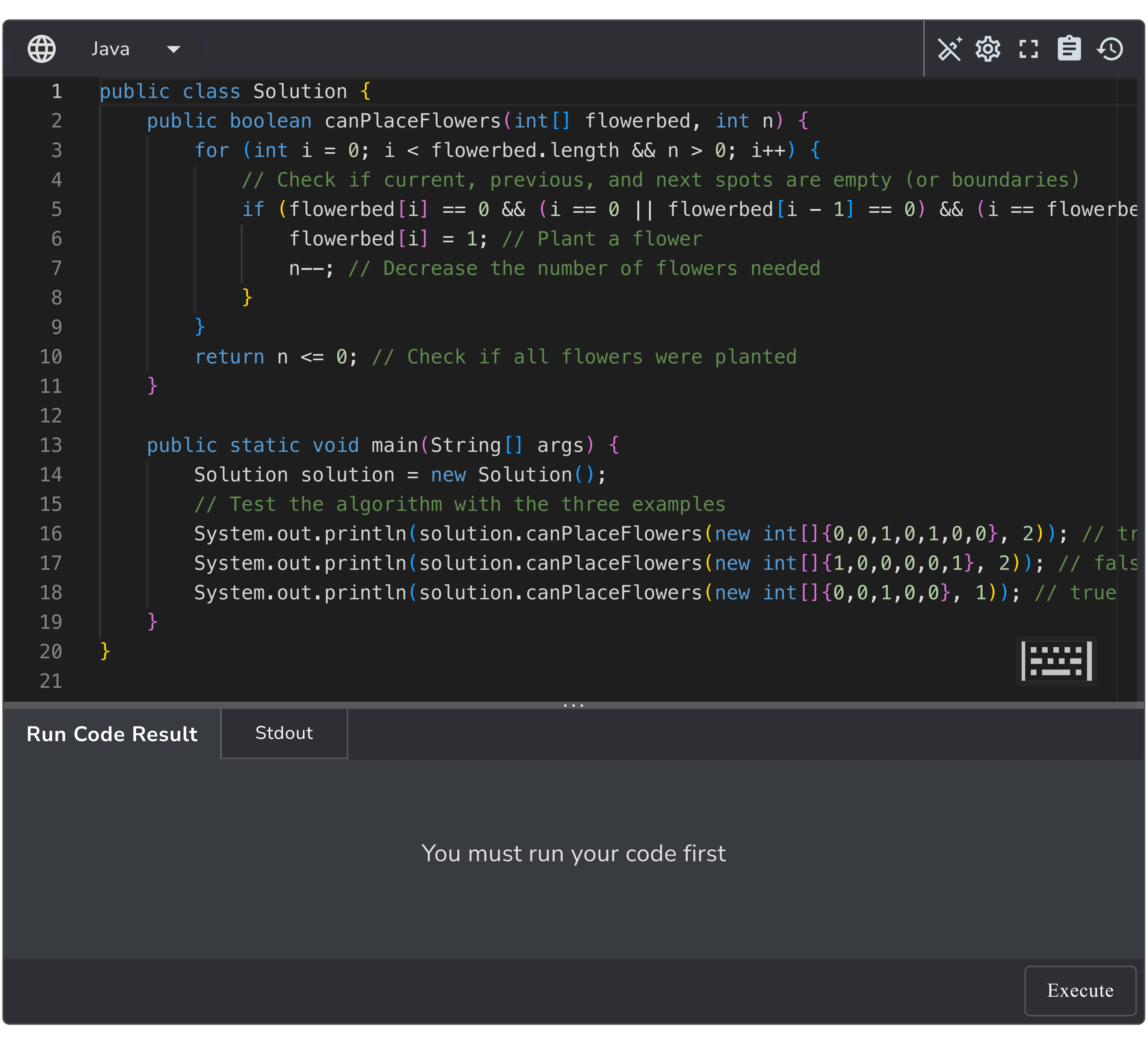
Algorithm Walkthrough

Let's consider the input `flowerbed = [0,0,1,0,1,0,0]`, `n = 2`.

1. **Initialization**:
   - Start with `flowerbed = [0,0,1,0,1,0,0]` and `n = 2`.
   - Initialize a counter for planting flowers (`n`), set to 2 initially, indicating we want to plant 2 flowers.
2. **Iteration Begins**:
   - Start iterating over the `flowerbed` array from the first element (index 0).
3. **First Element (Index 0)**:
   - Check the first element:
     - It is `0`, and since it's the first element, there's no left neighbor.
     - The right neighbor is also `0`.
   - According to the rule, a flower can be planted here because both the spot itself and its right neighbor are empty.
   - Update the flowerbed to `[1,0,1,0,1,0,0]` and decrease `n` to 1.
4. **Second Element (Index 1)**:
   - Now, the second element is `0`.
   - However, we just planted a flower at index 0, so we cannot plant another flower here due to the adjacency rule.
5. **Third Element (Index 2)**:
   - The element at index 2 is `1`, indicating a flower is already planted here, so move to the next element.
6. **Fourth Element (Index 3)**:
   - The element at index 3 is `0`.
   - Its left neighbor is `1` (flower planted), and its right neighbor is `0`.
   - We cannot plant a flower here because it would be adjacent to the flower at index 2.
7. **Fifth Element (Index 4)**:
   - The element at index 4 is `1`, so we move to the next element without doing anything.
8. **Sixth Element (Index 5)**:
   - The element at index 5 is `0`.
   - Its left neighbor is `1` (flower planted), and its right neighbor is `0`.
   - We cannot plant a flower here because it would be adjacent to the flower at index 4.
9. **Seventh Element (Index 6)**:
   - The element at index 6 is `0`.
   - This is the last element, so it has no right neighbor, and its left neighbor is `0`.
   - According to the rule, a flower can be planted here because both the spot itself and its left neighbor are empty.
   - Update the flowerbed to `[1,0,1,0,1,0,1]` and decrease `n` to 0.
10. **End of Iteration**:
    - We have finished iterating through the flowerbed array.
11. **Check Remaining Flowers**:
    - `n` is now `0`, indicating we have successfully planted all 2 flowers according to the rules.


Input: flowerbed = [0,0,1,0,1,0,0], n = 2
i = 0
1 0 1 0 1 0 0
i == 0, and flowerbed[i + 1] == 0. So, update flowerbed[i] = 1.
Update n = n - 1 = 2 - 1 = 1
i = 1
1 0 1 0 1 0 0
flowerbed[i - 1] != 0. So, continue to the next iteration.
i = 2
1 0 1 0 1 0 0
flowerbed[i] == 1. So, continue to the next iteration.
i = 3
1 0 1 0 1 0 0
flowerbed[i - 1] != 0. So, continue to the next iteration.
i = 4
1 0 1 0 1 0 0
flowerbed[i] == 1. So, continue to the next iteration.
i = 5
1 0 1 0 1 0 0
flowerbed[i - 1] != 0. So, continue to the next iteration.
i = 6
1 0 1 0 1 0 0
i ==flowerbed.length - 1 && flowerbed[i - 1] == 0.
So, update flowerbed[i] = 1.
Update n = n - 1 = 1 - 1 = 0.
Now, n <= 0. So, we can plant 2 flowers without breaking the given condition.

Code



```
public class Solution {
    public boolean canPlaceFlowers(int[] flowerbed, int n) {
        for (int i = 0; i < flowerbed.length && n > 0; i++) {
            // Check if current, previous, and next spots are empty (or boundaries)
            if (flowerbed[i] == 0 && (i == 0 || flowerbed[i - 1] == 0) && (i == flowerbe
                flowerbed[i] = 1; // Plant a flower
                n--; // Decrease the number of flowers needed
            }
        }
        return n <= 0; // Check if all flowers were planted
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
        // Test the algorithm with the three examples
        System.out.println(solution.canPlaceFlowers(new int[]{0,0,1,0,1,0,0}, 2)); // tr
        System.out.println(solution.canPlaceFlowers(new int[]{1,0,0,0,0,1}, 2)); // fals
        System.out.println(solution.canPlaceFlowers(new int[]{0,0,1,0,0}, 1)); // true
    }
}
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(n)$, where n is the length of the flowerbed array. This is because the algorithm iterates through the array once.
- **Space Complexity**: $O(1)$, indicating constant space usage regardless of the input size, as no additional space proportional to the input size is used.

← Previous
Can Place Flowers (Easy)

Next →
Reverse A LinkedList (Easy)

☐ Mark as Completed